[1부] 머신러닝
4장 모델 훈련

핸즈온 머신러닝(3판)

ㅇㅇ대학교 ㅇㅇ학과
홍길동

시작하기 전에

지은이: 오렐리앙 제롱 Aurélien Géron

머신러닝 컨설턴트. 2013년에서 2016년까지 구글에서 유튜브 동영상 분류 팀을 이끌었다. 2002년에서 2012년까지 프랑스의 모바일 ISP 선두주자인 위퍼스트(Wifirst)를 설립하고 CTO로 일했다. 2001년에는 폴리콘셀(Polyconseil)을 설립하고 CTO로 일했다. 이 회사는 지금 전기차 공유 서비스인 오토립(Autolib')을 운영하고 있다.
그전에는 재무(J. P. 모건과 소시에테 제네랄(Société Générale)), 방위(캐나다 국방부), 의료(수혈) 등 다양한 분야에서 엔지니어로 일했고, C++, WiFi, 인터넷 구조에 관한 몇 권의 기술 서적을 썼으며 한 프랑스 공과대학에서 컴퓨터과학을 가르쳤다.

옮긴이: 박해선 haesun.park@tensorflow.blog

기계공학을 전공했지만 졸업 후엔 줄곧 코드를 읽고 쓰는 일을 했다. 블로그(*tensorflow.blog*)에 글을 쓰고 머신러닝과 딥러닝에 관한 책을 집필, 번역하면서 소프트웨어와 과학의 경계를 흥미롭게 탐험하고 있다.
『챗GPT로 대화하는 기술』(한빛미디어, 2023), 『혼자 공부하는 머신러닝+딥러닝』(한빛미디어, 2020), 『혼자 공부하는 데이터 분석 with 파이썬』(한빛미디어, 2023), 『Do it! 딥러닝 입문』(이지스퍼블리싱, 2019) 등 집필했다.

[코드 예제] *https://bit.ly/homl3-git* 에서 주피터 노트북으로 제공

3판의 주요 변경 내용

1. 최신 라이브러리 버전으로 전체 코드 업데이트
2. 특성 이름 추적, 히스토그램 기반 그레이디언트 부스팅, 레이블 전파 등 사이킷런에 새롭게 추가된 다양한 기능
3. 하이퍼파라미터 튜닝을 위한 케라스 튜너(Keras Tuner) 라이브러리, 자연어 처리를 위한 허깅 페이스(Hugging Face)의 트랜스포머스(Transformers) 라이브러리 및 케라스의 새로운 전처리 및 데이터 증식 층
4. 여러 비전 모델(ResNeXt, DenseNet, MobileNet, CSPNet, EfficientNet)과 올바른 모델을 선택하기 위한 가이드라인
5. <15장 RNN과 CNN을 사용한 시퀀스 처리>: 합성된 시계열 대신 시카고 버스 및 철도 탑승객 데이터를 분석하며 ARMA 모델과 그 변형
6. <16장 RNN과 어텐션을 사용한 자연어 처리>: 인코더-디코더 RNN, 트랜스포머 모델을 사용한 영어-스페인어 번역 모델 구축, 스위치 트랜스포머(Switch Transformer), DistilBERT, T5, PaLM(사고 사슬 프롬프트 포함)과 같은 언어 모델 비전 트랜스포머(ViT) 소개, DeiT, 퍼시비어, DINO와 같은 트랜스포머 기반 비전 모델을 비롯한 CLIP, DALL·E, 플라밍고, GATO 등 몇 가지 대형 멀티모달 모델 개요
7. <17장 오토인코더, GAN 그리고 확산 모델>: 확산 모델 소개, DDPM 구현
8. <19장 대규모 텐서플로 모델 훈련과 배포>: 구글 클라우드 AI 플랫폼에서 구글 버텍스 AI로 마이그레이션, 대규모 하이퍼파라미터 검색을 위한 분산 케라스 튜너, TensorFlow.js 코드
9. PipeDream과 Pathways를 비롯한 추가적인 분산 훈련 기법

[참조] https://homl.info/changes3

이 책의 학습 목표

1부 머신러닝

1장: 한눈에 보는 머신러닝	데이터 과학자가 꼭 알아야 할 기초 개념과 용어
2장: 머신러닝 프로젝트 처음부터 끝까지	주택 가격을 예측하는 회귀 작업을 살펴보면서 선형 회귀, 결정 트리, 랜덤 포레스트 등 여러 알고리즘 학습
3장: 분류	분류 시스템 학습
4장: 모델 훈련	신경망 구축에 필요한 모델 훈련 알고리즘 학습
5장: 서포트 벡터 머신	SVM의 핵심 개념, 사용 방법, 작동 원리 학습
6장: 결정 트리	결정 트리의 훈련, 시각화, 예측 방법과 사이킷런의 CART 훈련 알고리즘 등
7장: 앙상블 학습과 랜덤 포레스트	투표 기반 분류기, 배깅과 페이스팅 앙상블, 랜덤 포레스트, 부스팅, 스태킹 앙상블 등
8장: 차원 축소	고차원 공간과 차원 축소 기법
9장: 비지도 학습	비지도 학습과 알고리즘

이 책의 학습 목표

2부 신경망과 딥러닝

10장: 케라스를 사용한 인공 신경망 소개	인공 신경망과 케라스를 이용한 구현 방법
11장: 심층 신경망 훈련	심층 신경망의 문제와 해결 방법
12장: 텐서플로를 사용한 사용자 정의 모델과 훈련	텐서플로와 저수준 파이썬 API를 이용한 사용자 정의 모델과 훈련 알고리즘
13장: 텐서플로를 사용한 데이터 적재와 전처리	tf.data API와 TFRecord 포맷, 케라스 전처리 층 및 tf.data API 사용 방법
14장: 합성곱 신경망을 사용한 컴퓨터 비전	CNN 구성 요소, 케라스를 사용한 구현 방법
15장: RNN과 CNN을 사용한 시퀀스 처리	RNN 개념과 WaveNet 구현
16장: RNN과 어텐션을 사용한 자연어 처리	문장 수준의 RNN과 어텐션 메커니즘
17장: 오토인코더, GAN 그리고 확산 모델	오토인코더 차원 축소, 특성 추출, 비지도 사전 훈련 방법과 생성 모델
18장: 강화 학습	강화 학습 개념, 정책 그레이디언트, 심층 Q-네트워크
19장: 대규모 텐서플로 모델 훈련과 배포	TF 서빙과 구글 버텍스 AI 플랫폼에 모델을 배포하는 방법

이 책의 학습 목표

3부 부록

부록 A: 연습문제 정답

부록 B: 머신러닝 프로젝트 체크리스트

부록 C: 자동 미분

부록 D: 특수한 데이터 구조

부록 E: 텐서플로 그래프

Contents

- 4장: 모델 훈련

4.1 선형 회귀
4.2 경사 하강법
4.3 다항 회귀
4.4 학습 곡선
4.5 규제가 있는 선형 모델
4.6 로지스틱 회귀

4장 모델 훈련

신경망 구축에 필요한 모델 훈련 알고리즘 학습

4.1 선형 회귀(1)

선형 회귀 모델

- 일반적으로 선형 모델은 입력 특성의 가중치 합과 편향(bias) 또는 절편(intercept)이라는 상수를 더해 예측을 만듦

$$\text{삶의 만족도} = \theta_0 + \theta_1 \times \text{1인당_GDP}$$

- 입력 특성인 1인당_GDP에 대한 선형 함수
- θ_0과 θ_1 모델 파라미터

식 4-1 선형 회귀 모델의 예측

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

- $\hat{y}$은 예측값
- n은 특성의 수
- x_i는 i번째 특성값
- θ_j는 j번째 모델 파라미터
  (편향 θ_0과 특성의 가중치 $\theta_1, \theta_2, \ldots, \theta_n$을 포함)

식 4-2 선형 회귀 모델의 예측(벡터 형태)

$$\hat{y} = h_{\boldsymbol{\theta}}(\mathbf{x}) = \boldsymbol{\theta} \cdot \mathbf{x}$$

- h_θ는 모델 파라미터 θ를 사용한 가설 함수
- θ는 편향 θ_0과 θ_1에서 θ_n까지의 특성 가중치를 담은 모델의 파라미터 벡터
- x는 x_0에서 x_n까지 담은 샘플의 특성 벡터
  x_0 은 항상 1
- θ · x는 벡터 θ와 x의 점곱
  이는 $\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \ldots + \theta_n x_n$ 과 같음

4.1 선형 회귀(2)

- 선형 회귀 모델의 훈련
  - 모델 훈련은 모델이 훈련 세트에 가장 잘 맞도록 모델 파라미터를 설정하는 것
    - 먼저 모델이 훈련 데이터에 얼마나 잘 들어맞는지 측정
    - 회귀에 가장 널리 사용되는 성능 측정 지표는 평균 제곱근 오차(RMSE)
    - 선형 회귀 모델을 훈련시키려면 RMSE를 최소화하는 θ를 찾아야 함
    - 실제로는 RMSE보다 평균 제곱 오차(mean square error, MSE)를 최소화하는 것이 같은 결과를 내면서 더 간단함

식 4-3 선형 회귀 모델의 MSE 비용 함수

$$\mathrm{MSE}(\mathbf{X}, h_{\boldsymbol{\theta}}) = \frac{1}{m}\sum_{i=1}^{m}\left(\boldsymbol{\theta}^{\mathrm{T}}\mathbf{x}^{(i)} - y^{(i)}\right)^2$$

4.1 선형 회귀(3)

4.1.1 정규 방정식

- 정규 방정식(normal equation)
  - 비용 함수를 최소화하는 θ 값을 찾기 위한 해석적인 방법

식 4-4 정규 방정식

$$\hat{\boldsymbol{\theta}} = \left(\mathbf{X}^{\mathrm{T}}\mathbf{X}\right)^{-1}\mathbf{X}^{\mathrm{T}}\mathbf{y}$$

- $\hat{\theta}$ 은 비용 함수를 최소화하는 θ 값입니다.
- y는 $y^{(1)}$부터 $y_{(m)}$까지 포함하는 타깃 벡터

4.1 선형 회귀(4)

- 정규 방정식을 테스트하기 위해 임의의 선형 데이터를 생성

```
import numpy as np

np.random.seed(42) # 동일하게 재현되도록 하기 위해 지정합니다.
m = 100                              # 샘플 개수
X = 2 * np.random.rand(m, 1)         # 열 벡터
y = 4 + 3 * X + np.random.randn(m, 1) # 열 벡터
```

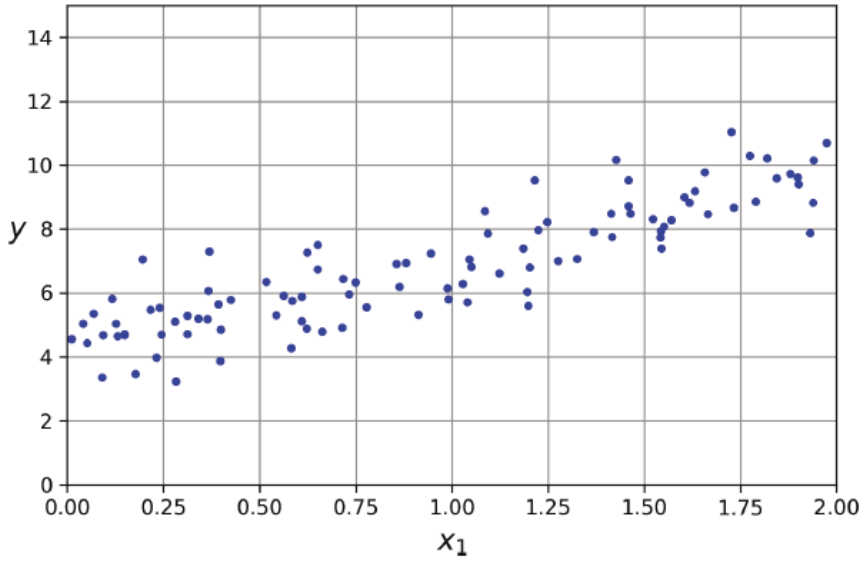

그림 4-1 랜덤으로 생성한 선형 데이터셋

4.1 선형 회귀(5)

- 정규 방정식을 사용해 $\hat{\theta}$을 계산
  - 넘파이 선형대수 모듈(np.linalg)의 inv() 함수를 사용해 역행렬을 계산
  - dot() 메서드를 사용해 행렬 곱셈 수행

```
from sklearn.preprocessing import add_dummy_feature

X_b = add_dummy_feature(X) # 각 샘플에 x0 = 1을 추가합니다.
theta_best = np.linalg.inv(X_b.T @ X_b) @ X_b.T @ y
```

  - 정규 방정식으로 계산한 값을 확인

```
>>> theta_best
array([[4.21509616],
       [2.77011339]])
```

4.1 선형 회귀(6)

- $\hat{\theta}$을 사용해 예측하고 그래프 그리기

```
>>> X_new = np.array([[0], [2]])
>>> X_new_b = add_dummy_feature(X_new) # 각 샘플에 x0 = 1을 추가합니다.
>>> y_predict = X_new_b @ theta_best
>>> y_predict
array([[4.21509616],
       [9.75532293]])
```

```
import matplotlib.pyplot as plt

plt.plot(X_new, y_predict, "r-", label="예측")
plt.plot(X, y, "b.")
[...]  # 레이블, 축, 그리드, 범례를 추가합니다.
plt.show()
```

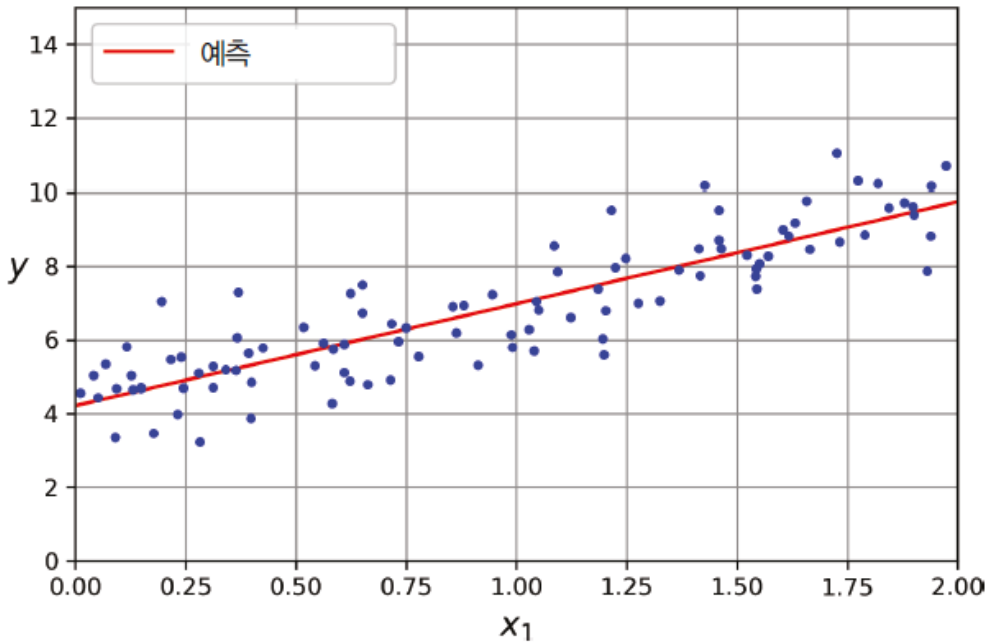

그림 4-2 선형 회귀 모델의 예측

4.1 선형 회귀(7)

- 사이킷런에서 선형 회귀를 수행
  - 사이킷런은 특성의 가중치(coef_)와 편향(intercept_)을 분리하여 저장

```
>>> from sklearn.linear_model import LinearRegression
>>> lin_reg = LinearRegression()
>>> lin_reg.fit(X, y)
>>> lin_reg.intercept_, lin_reg.coef_
(array([4.21509616]), array([[2.77011339]]))
>>> lin_reg.predict(X_new)
array([[4.21509616],
       [9.75532293]])
```

  - LinearRegression 클래스는 scipy.linalg.lstsq() 함수를 직접 호출
    - 이 함수는 $\hat{\theta} = X^{+}y$를 계산
      - X^{+}는 X의 유사역행렬(pseudoinverse)
      - 무어-펜로즈(Moore-Penrose) 유사역행렬

```
>>> theta_best_svd, residuals, rank, s = np.linalg.lstsq(X_b, y, rcond=1e-6)
>>> theta_best_svd
array([[4.21509616],
       [2.77011339]])
```

4.1 선형 회귀(8)

- np.linalg.pinv() 함수를 사용해 유사역행렬을 직접 구하기

```
>>> np.linalg.pinv(X_b) @ y
array([[4.21509616],
       [2.77011339]])
```

- 유사역행렬 자체는 특잇값 분해(singular value decomposition, SVD)라 부르는 표준 행렬 분해 기법을 사용해 계산
  - SVD는 훈련 세트 행렬 X를 3개의 행렬 곱셈 $U\Sigma V^T$로 분해 (numpy.linalg.svd()를 참고)
  - 유사역행렬은 $X^+=V\Sigma^+U^T$로 계산

4.1 선형 회귀(9)

4.1.2 계산 복잡도

- 정규 방정식은 $(n + 1) \times (n + 1)$ 크기의 X^TX의 역행렬을 계산(n은 특성 수)
- 역행렬을 계산하는 계산 복잡도(computational complexity)는 일반적으로 $O(n^{2.4})$에서 $O(n^3)$ 사이
  - 특성 수가 두 배로 늘어나면 계산 시간이 대략 $2^{2.4} = 5.3$에서 $2^3 = 8$배로 증가
- 사이킷런의 LinearRegression 클래스가 사용하는 SVD 방법은 약 $O(n^2)$

4.2 경사 하강법(1)

- 경사 하강법(gradient descent, GD)
  - 여러 종류의 문제에서 최적의 해법을 찾을 수 있는 일반적인 최적화 알고리즘
    - 빨리 골짜기로 내려가는 좋은 방법은 가장 가파른 길을 따라 아래로 내려가는 것
    - 파라미터 벡터 θ에 대해 비용 함수의 현재 그레이디언트(gradient)를 계산
    - 그레이디언트가 감소하는 방향으로 진행하여 그레이디언트가 0이 되면 최솟값에 도달

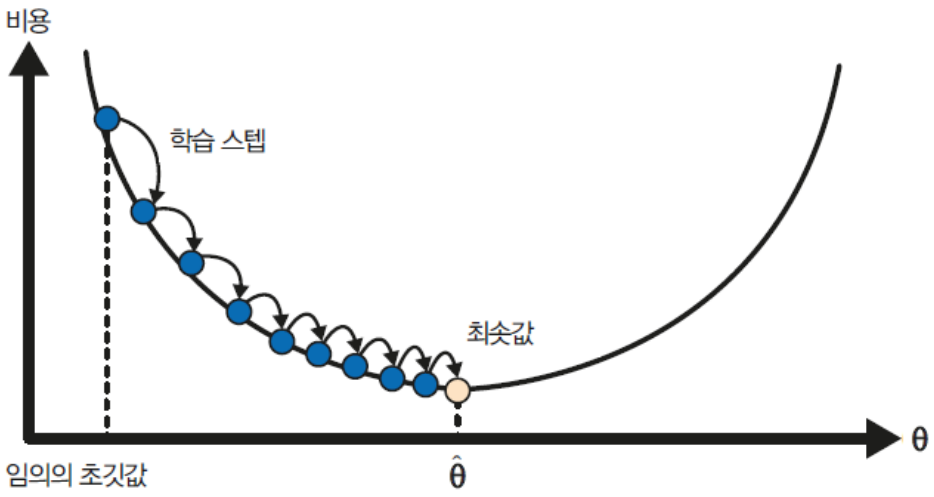

그림 4-3 여기서 모델 파라미터는 랜덤하게 초기화된 후 반복적으로 수정되어 비용 함수를 최소화
학습 스텝 크기는 비용 함수의 기울기에 비례
따라서 비용이 최솟값에 가까워질수록 스텝 크기가 점진적으로 줄어듬

4.2 경사 하강법(2)

- 학습률 하이퍼파라미터
  - 학습률이 너무 작으면 알고리즘이 수렴하기 위해 반복을 많이 진행해야 하므로 시간이 오래 걸림
  - 학습률이 너무 크면 알고리즘을 더 큰 값으로 발산하게 만들어 적절한 해법을 찾지 못함

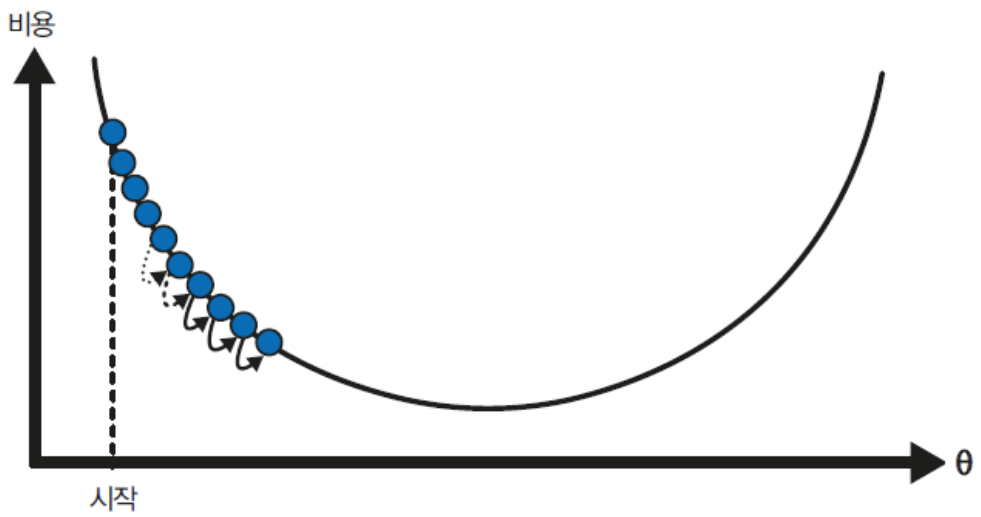

그림 4-4 학습률이 너무 작을 때

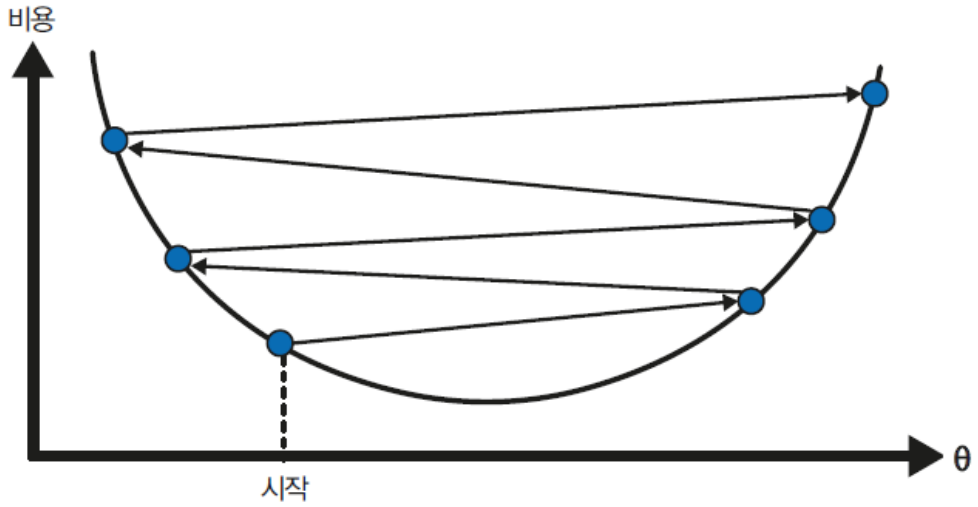

그림 4-5 학습률이 너무 클 때

4.2 경사 하강법(3)

- 경사 하강법의 두 가지 문제점
  - 알고리즘이 왼쪽에서 시작하면 전역 최솟값(global minimum)보다 덜 좋은 지역 최솟값(local minimum)에 수렴
  - 오른쪽에서 시작하면 평탄한 지역을 지나기 위해 시간이 오래 걸리고 일찍 멈추게 되어 전역 최솟값에 도달하지 못함
  - 다행히 선형 회귀를 위한 MSE 비용 함수는 곡선에서 어떤 두 점을 선택해 선을 그어도 곡선을 가로지르지 않는 볼록 함수(convex function)

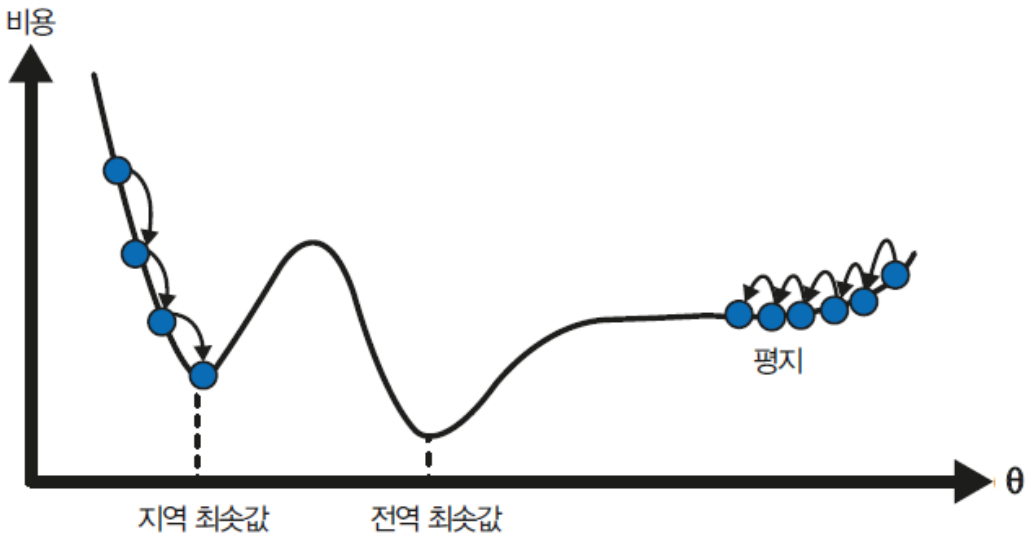

그림 4-6 경사 하강법의 문제점

4.2 경사 하강법(4)

- 특성들의 스케일이 매우 다르면 길쭉한 모양일 수 있음
  - [그림 4-7]은 특성 1과 특성 2의 스케일이 같은 훈련 세트(왼쪽), 특성 1이 특성 2보다 더 작은 훈련 세트(오른쪽)
  - 모델 훈련은 (훈련 세트에서) 비용 함수를 최소화하는 모델 파라미터의 조합을 찾는 일
    - 모델의 파라미터 공간(parameter space)에서 찾는다고 표현

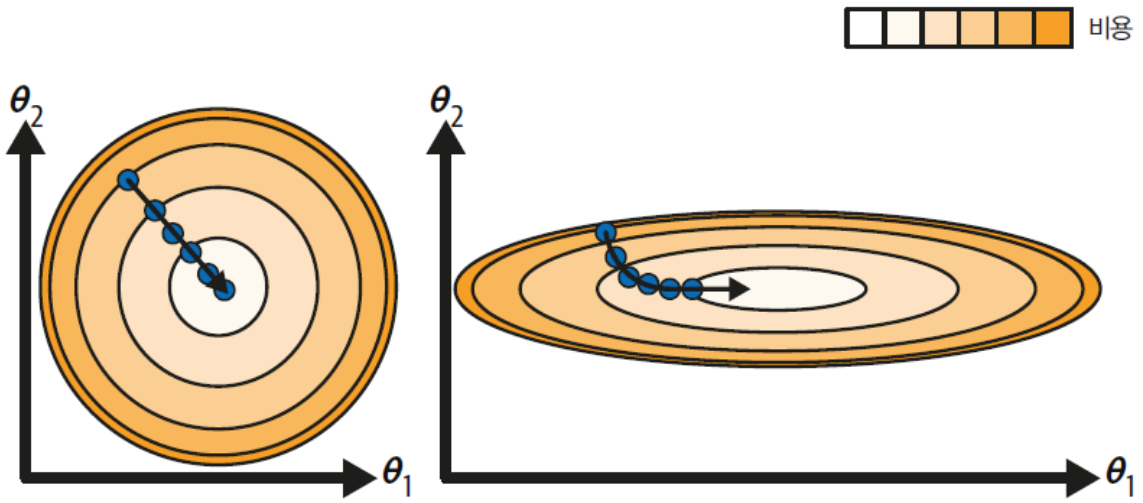

그림 4-7 특성 스케일을 적용한 경사 하강법(왼쪽)과 적용하지 않은 경사 하강법(오른쪽

4.2 경사 하강법(5)

4.2.1 배치 경사 하강법

- 편도함수(partial derivative)
  - θ_j가 조금 변경될 때 비용 함수가 얼마나 바뀌는지 계산

식 4-5 비용 함수의 편도함수

$$\frac{\partial}{\partial \theta_j} \mathrm{MSE}(\boldsymbol{\theta}) = \frac{2}{m} \sum_{i=1}^{m} \left(\boldsymbol{\theta}^{\mathrm{T}} \mathbf{x}^{(i)} - y^{(i)} \right) x_j^{(i)}$$

  - 그레이디언트 벡터 - 편도함수를 각각 계산하는 대신 [식 4-6]을 사용하여 한꺼번에 계산

식 4-6 비용 함수의 그레이디언트 벡터

$$\nabla_{\boldsymbol{\theta}} \mathrm{MSE}(\boldsymbol{\theta}) = \begin{pmatrix} \frac{\partial}{\partial \theta_0} \mathrm{MSE}(\boldsymbol{\theta}) \\ \frac{\partial}{\partial \theta_1} \mathrm{MSE}(\boldsymbol{\theta}) \\ \vdots \\ \frac{\partial}{\partial \theta_n} \mathrm{MSE}(\boldsymbol{\theta}) \end{pmatrix} = \frac{2}{m} \mathbf{X}^{\mathrm{T}} (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})$$

4.2 경사 하강법(6)

- 내려가는 스텝의 크기를 결정하기 위해 그레이디언트 벡터에 학습률 η(에타 eta)를 곱함

식 4-7 경사 하강법의 스텝 $\boldsymbol{\theta}^{(\text{next step})} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \text{MSE}(\boldsymbol{\theta})$

```
eta = 0.1    # 학습률
n_epochs = 1000
m = len(X_b) # 샘플 개수

np.random.seed(42)
theta = np.random.randn(2, 1) # 모델 파라미터를 랜덤하게 초기화합니다.

for epoch in range(n_epochs):
    gradients = 2 / m * X_b.T @ (X_b @ theta - y)
    theta = theta - eta * gradients
```

- 에포크(epoch) - 훈련 세트를 한 번 반복하는 것

```
>>> theta
array([[4.21509616],
       [2.77011339]])
```

4.2 경사 하강법(7)

- 세 가지 학습률을 사용하여 진행한 경사 하강법의 첫 20 스텝
  - 적절한 학습률을 찾기 위해 그리드 서치를 사용
    - 그리드 서치에서 수렴하는 데 너무 오래 걸리는 모델이 제외될 수 있도록 반복 횟수를 제한
  - 반복 횟수
    - 너무 작으면 최적점에 도달하기 전에 알고리즘이 멈춤
    - 너무 크면 모델 파라미터가 더는 변하지 않는 동안 시간을 낭비
    - 간단한 해결책은 반복 횟수를 아주 크게 지정하고 그레이디언트 벡터가 아주 작아지면, 즉 벡터의 노름이 어떤 값 ε(허용 오차(tolerance))보다 작아지면 경사 하강법이 (거의) 최솟값에 도달한 것이므로 알고리즘을 중지

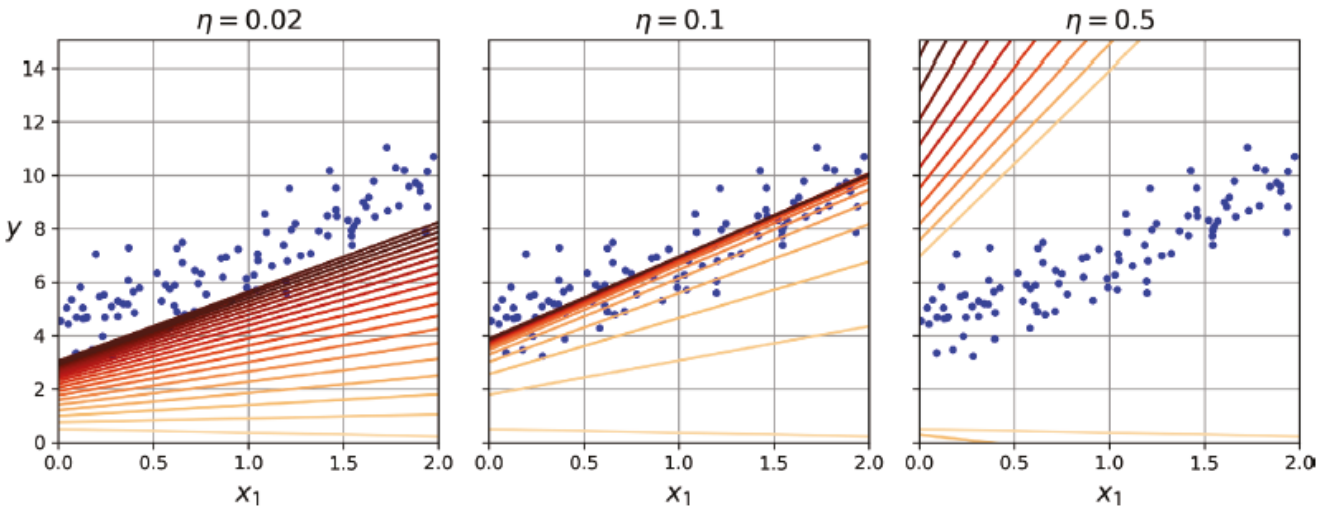

그림 4-8 여러 가지 학습률에 대한 경사 하강법

4.2 경사 하강법(8)

4.2.2 확률적 경사 하강법

- 확률적 경사 하강법은 매 스텝에서 한 개의 샘플을 랜덤으로 선택, 그 하나의 샘플에 대한 그레이디언트를 계산
  - 한 번에 하나의 샘플을 처리하면 매 반복에서 다뤄야 할 데이터가 매우 적기 때문에 알고리즘이 확실히 훨씬 빠름
  - 확률적(랜덤)이므로 이 알고리즘은 배치 경사 하강법보다 훨씬 불안정
- 학습 스케줄(learning schedule) - 매 반복에서 학습률을 결정하는 함수

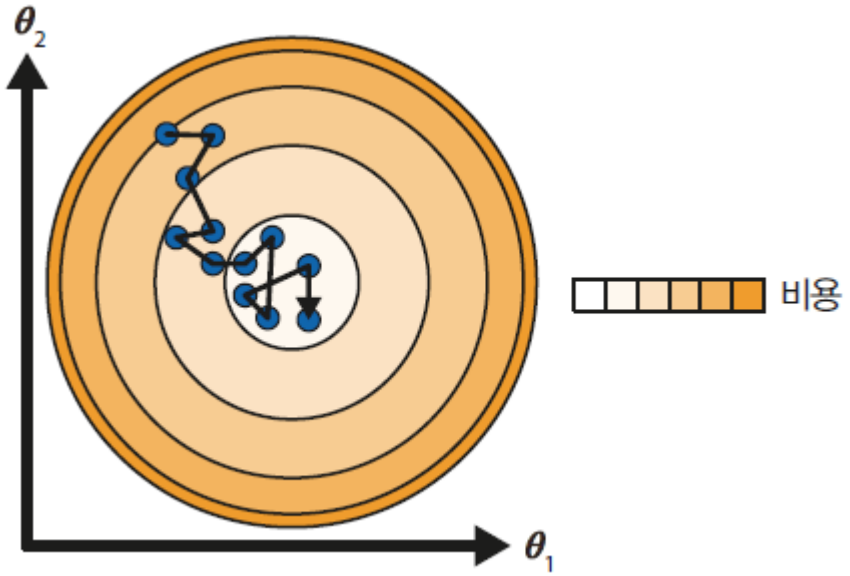

그림 4-9 확률적 경사 하강법을 사용하면 개별 훈련 스텝은 매우 빠르지만 배치 경사 하강법을 사용할 때보다 훨씬 더 확률적

4.2 경사 하강법(9)

- 학습 스케줄을 사용한 확률적 경사 하강법의 구현

```
n_epochs = 50
t0, t1 = 5, 50 # 학습 스케줄 하이퍼파라미터

def learning_schedule(t):
    return t0 / (t + t1)

np.random.seed(42)
theta = np.random.randn(2, 1) # 랜덤 초기화

for epoch in range(n_epochs):
    for iteration in range(m):
        random_index = np.random.randint(m)
        xi = X_b[random_index:random_index + 1]
        yi = y[random_index:random_index + 1]
        gradients = 2 * xi.T @ (xi @ theta - yi) # SGD의 경우 m으로 나누지 않습니다.
        eta = learning_schedule(epoch * m + iteration)
        theta = theta - eta * gradients
```

```
>>> theta
array([[4.21076011],
       [2.74856079]])
```

4.2 경사 하강법(10)

- 훈련의 첫 20 스텝
  - 스텝이 불규칙하게 진행
  - 샘플을 랜덤으로 선택하므로 어떤 샘플은 한 에포크에서 여러 번 선택될 수 있고 어떤 샘플은 전혀 선택되지 못할 수 있음

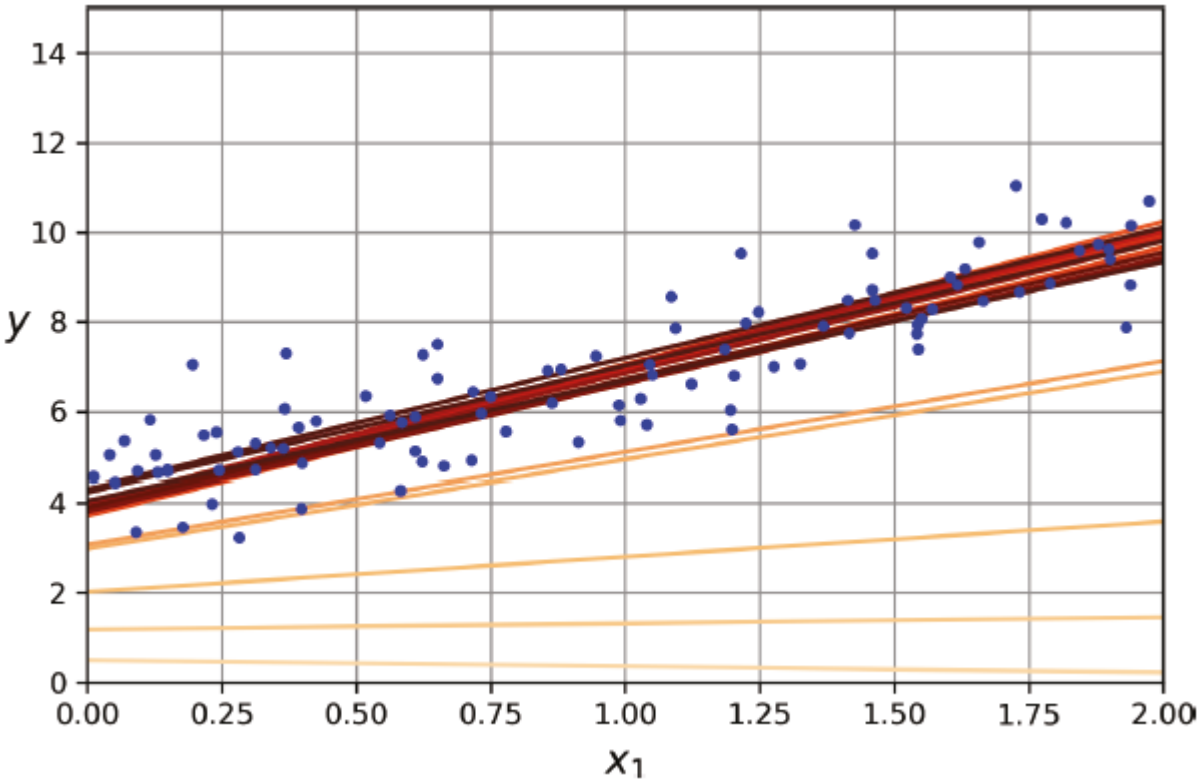

그림 4-10 확률적 경사 하강법의 첫 20 스텝

4.2 경사 하강법(11)

- SGDRegressor 클래스
  - 사이킷런에서 SGD 방식으로 제곱 오차 비용 함수를 최적화하는 선형 회귀

```
from sklearn.linear_model import SGDRegressor

sgd_reg = SGDRegressor(max_iter=1000, tol=1e-5, penalty=None, eta0=0.01,
                       n_iter_no_change=100, random_state=42)
sgd_reg.fit(X, y.ravel()) # fit()이 1D 타깃을 기대하기 때문에 y.ravel()로 씁니다.
```

```
>>> sgd_reg.intercept_, sgd_reg.coef_
(array([4.21278812]), array([2.77270267]))
```

4.2 경사 하강법(12)

4.2.3 미니배치 경사 하강법

- 미니배치 경사 하강법(mini-batch gradient descent)
  - 미니배치라 부르는 임의의 작은 샘플 세트에 대해 그레이디언트를 계산
  - 장점 - 확률적 경사 하강법에 비해 행렬 연산에 최적화된 하드웨어, 특히 GPU를 사용해서 성능을 향상

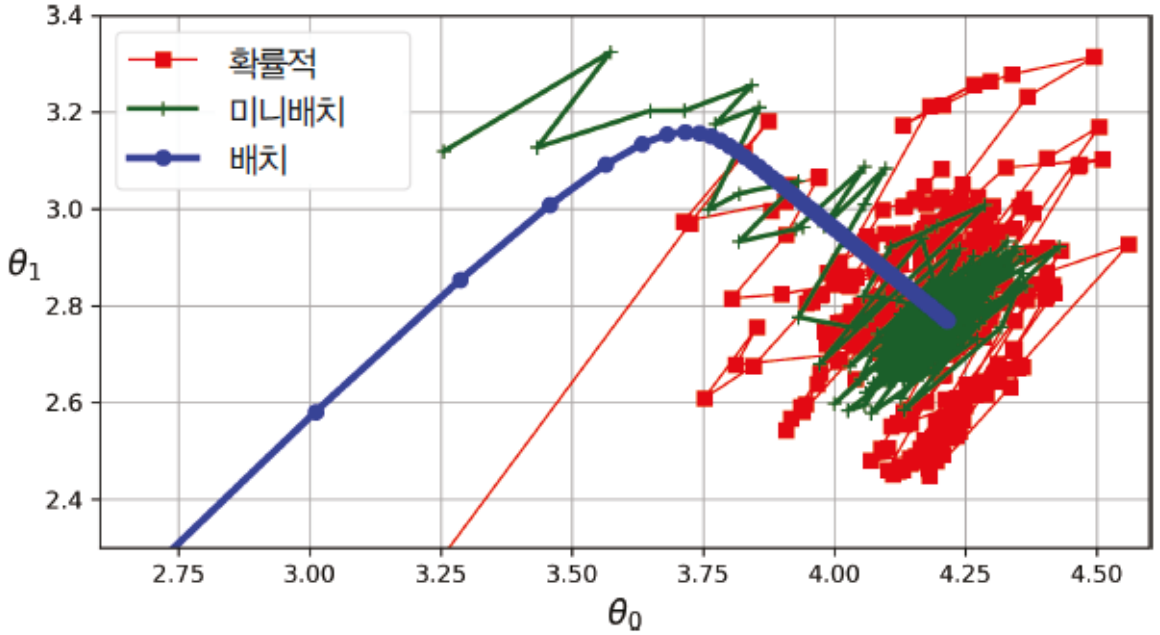

그림 4-11 파라미터 공간에 표시된 경사 하강법의 경로

4.2 경사 하강법(13)

- 최적화 알고리즘 비교
  - *m*(훈련 샘플 수), *n*(특성 수)

표 4-1 선형 회귀를 사용한 알고리즘 비교

알고리즘	*m*이 클 때	외부 메모리 학습 지원	*n*이 클 때	하이퍼 파라미터 수	스케일 조정 필요	사이킷런
정규 방정식	빠름	No	느림	0	No	N/A
SVD	빠름	No	느림	0	No	LinearRegression
배치 경사 하강법	느림	No	빠름	2	Yes	N/A
확률적 경사 하강법	빠름	Yes	빠름	≥ 2	Yes	SGDRegressor
미니배치 경사 하강법	빠름	Yes	빠름	≥ 2	Yes	N/A

4.3 다항 회귀(1)

- 다항 회귀(polynomial regression)
  - 비선형 데이터에 대해 각 특성의 거듭제곱을 새로운 특성으로 추가하고, 이 확장된 특성을 포함한 데이터셋에 선형 모델을 훈련시키는 것
  - y = ax2 + bx + c인 간단한 2차 방정식에 약간의 잡을 추가한 비선형 데이터를 준비

```
np.random.seed(42)
m = 100
X = 6 * np.random.rand(m, 1) - 3
y = 0.5 * X ** 2 + X + 2 + np.random.randn(m, 1)
```

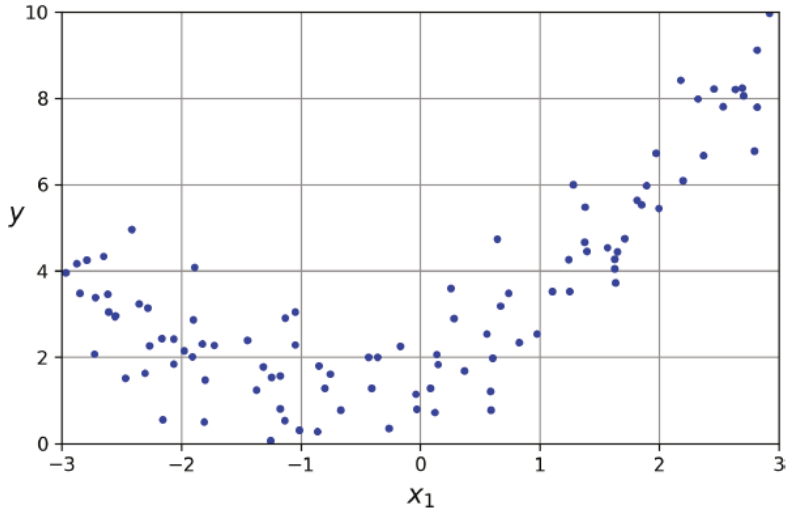

그림 4-12 잡음이 포함된 비선형 데이터셋

4.3 다항 회귀(2)

- 사이킷런의 PolynomialFeatures를 사용해 훈련 데이터를 변환
  - 훈련 세트에 있는 각 특성을 제곱(2차 다항)하여 새로운 특성으로 추가
  - X_poly는 이제 원래 특성 X와 이 특성의 제곱을 포함

```
>>> from sklearn.preprocessing import PolynomialFeatures
>>> poly_features = PolynomialFeatures(degree=2, include_bias=False)
>>> X_poly = poly_features.fit_transform(X)
>>> X[0]
array([-0.75275929])
>>> X_poly[0]
array([-0.75275929, 0.56664654])
```

  - 확장된 훈련 데이터에 LinearRegression을 적용

```
>>> lin_reg = LinearRegression()
>>> lin_reg.fit(X_poly, y)
>>> lin_reg.intercept_, lin_reg.coef_
(array([1.78134581]), array([[0.93366893, 0.56456263]]))
```

4.3 다항 회귀(3)

- 특성이 여러 개일 때 다항 회귀는 특성 사이의 관계를 찾을 수 있음
  - 일반적인 선형 회귀 모델에서는 불가능
  - PolynomialFeatures가 주어진 차수까지 특성 간의 모든 교차항을 추가하기 때문임

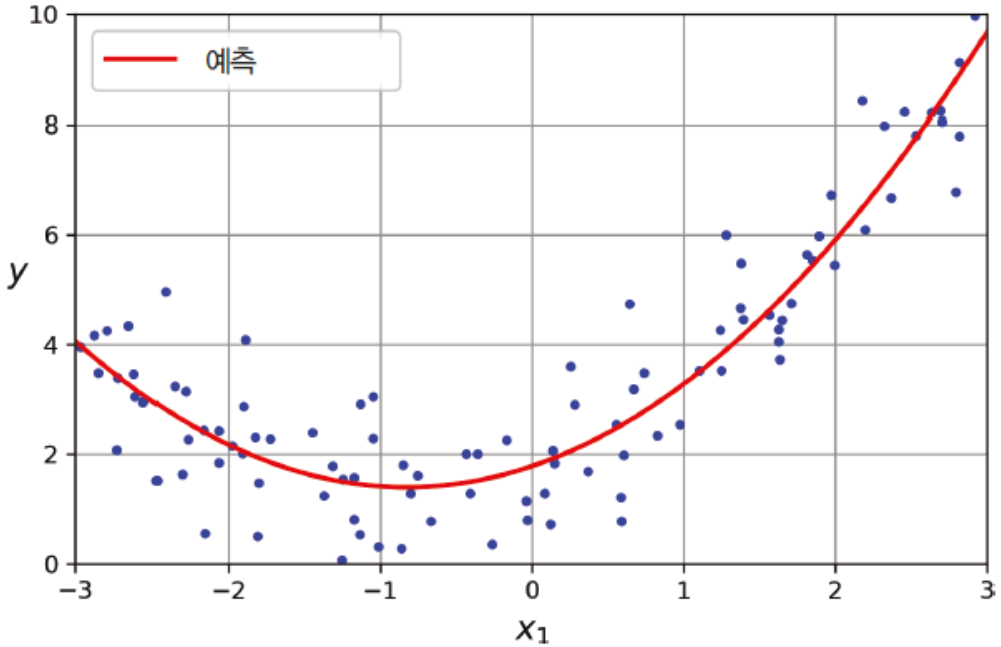

그림 4-13 다항 회귀 모델의 예측

- 원래 함수 $y = 0.5x_1^2 + 1.0x_1 + 2.0$ +가우스_잡음
- 예측된 모델 $\hat{y} = 0.56x_1^2 + 0.93x_1 + 1.78$

4.4 학습 곡선(1)

- 300차 다항 회귀 모델을 이전 훈련 데이터에 적용
  - 이 고차 다항 회귀 모델은 심각하게 훈련 데이터에 과대적합
  - 반면에 선형 모델은 과소적합
  - 이 경우 가장 일반화가 잘되는 모델은 2차 다항 회귀

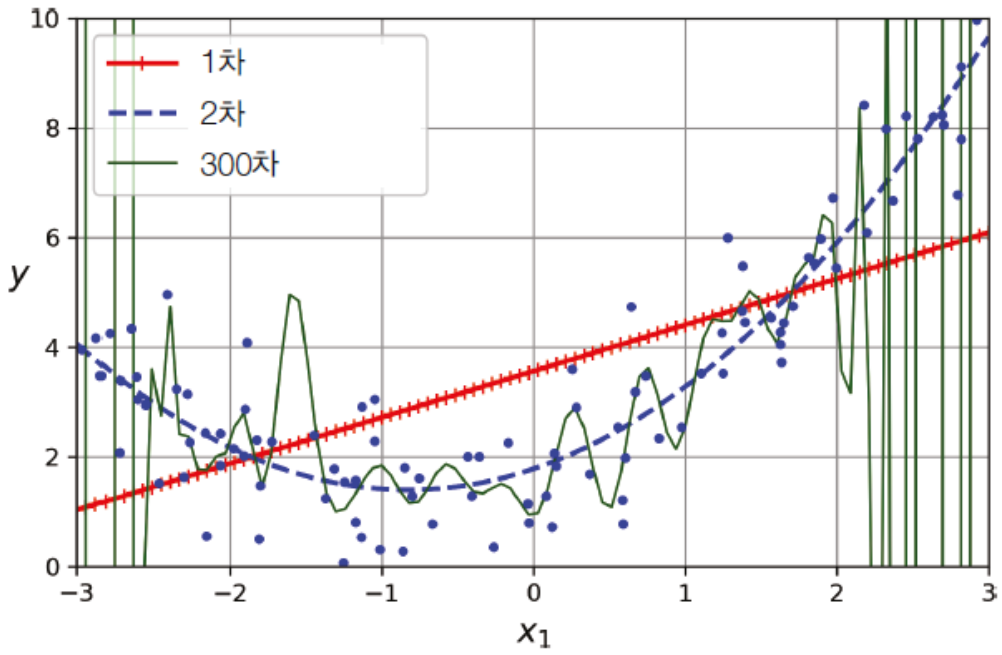

그림 4-14 고차 다항 회귀

4.4 학습 곡선(2)

- learning_curve() 함수
  - 교차 검증을 사용하여 모델을 훈련하고 평가
  - 결과는 과소적합

```
from sklearn.model_selection import learning_curve

train_sizes, train_scores, valid_scores = learning_curve(
    LinearRegression(), X, y, train_sizes=np.linspace(0.01, 1.0, 40), cv=5,
    scoring="neg_root_mean_squared_error")
train_errors = -train_scores.mean(axis=1)
valid_errors = -valid_scores.mean(axis=1)

plt.plot(train_sizes, train_errors, "r-+", linewidth=2, label="훈련 세트")
plt.plot(train_sizes, valid_errors, "b-", linewidth=3, label="검증 세트")
[...] # 레이블, 축, 그리드, 범례를 추가합니다.
plt.show()
```

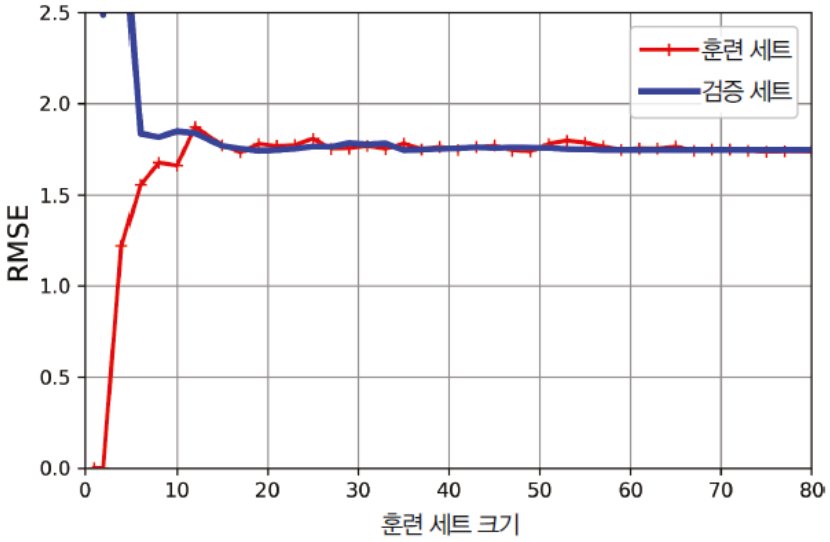

그림 4-15 학습 곡선

4.4 학습 곡선(3)

- 10차 다항 회귀 모델의 학습 곡선

```
from sklearn.pipeline import make_pipeline

polynomial_regression = make_pipeline(
    PolynomialFeatures(degree=10, include_bias=False),
    LinearRegression())

train_sizes, train_scores, valid_scores = learning_curve(
    polynomial_regression, X, y, train_sizes=np.linspace(0.01, 1.0, 40), cv=5,
    scoring="neg_root_mean_squared_error")
[...] # 이전과 동일
```

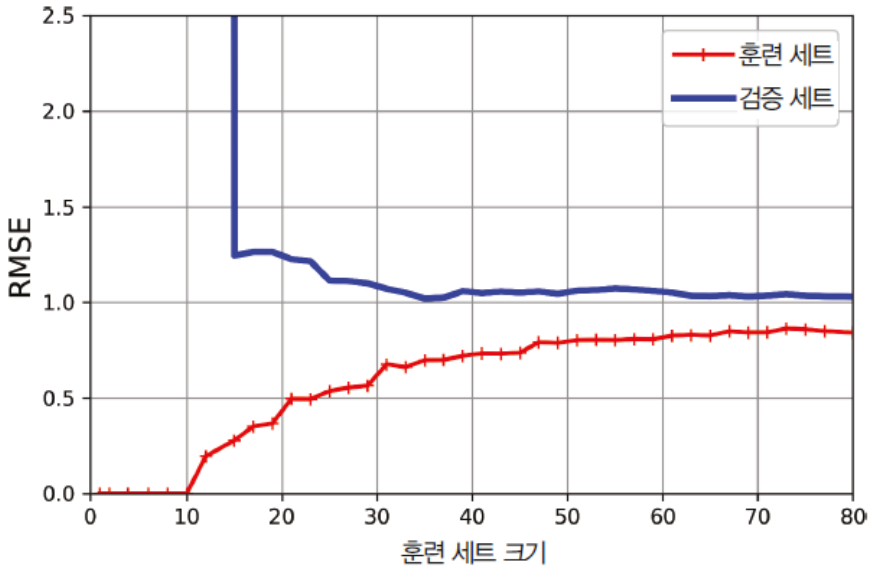

그림 4-16 10차 다항 회귀의 학습 곡선

- 훈련 데이터의 오차가 이전보다 훨씬 낮음
- 두 곡선 사이에 공간이 있음
  - 검증 데이터에서보다 훈련 데이터에서 모델이 훨씬 더 나은 성능을 보인다는 의미
  - 과대적합 모델의 특징
  - 그러나 더 큰 훈련 세트를 사용하면 두 곡선이 점점 가까워짐

4.4 학습 곡선(4)

편향/분산 트레이드오프

- **편향**

  일반화 오차 중에서 편향은 잘못된 가정으로 인한 것

  예를 들어 데이터가 실제로는 2차인데 선형으로 가정하는 경우

  편향이 큰 모델은 훈련 데이터에 과소적합되기 쉬움

- **분산**

  분산은 훈련 데이터에 있는 작은 변동에 모델이 과도하게 민감하기 때문에 나타남

  자유도가 높은 모델(예 고차 다항 회귀 모델)이 높은 분산을 가지기 쉬워 훈련 데이터에 과대적합되는 경향

- **줄일 수 없는 오차**

  줄일 수 없는 오차(irreducible error)는 데이터 자체에 있는 잡음 때문에 발생

  이 오차를 줄일 수 있는 유일한 방법은 데이터에서 잡음을 제거하는 것

  (예를 들어 고장 난 센서 같은 데이터 소스를 고치거나 이상치를 감지해 제거)

4.5 규제가 있는 선형 모델(1)

4.5.1 릿지 회귀

- 릿지 회귀(ridge regression) 또는 티호노프 규제(Tikhonov regularization)
  - 규제가 추가된 선형 회귀 버전
  - 규제항 $\frac{a}{m}\Sigma_{i=1}^{n}\theta_i^2$ 이 MSE에 추가

식 4-8 릿지 회귀의 비용 함수

$$J(\boldsymbol{\theta}) = \mathrm{MSE}(\boldsymbol{\theta}) + \frac{\alpha}{m}\sum_{i=1}^{n}\theta_i^2$$

4.5 규제가 있는 선형 모델(2)

- 잡음이 많은 선형 데이터에 몇 가지 다른 α를 사용해 릿지 모델을 훈련시킨 결과 비교
  - (왼쪽 그래프) 평범한 릿지 모델을 사용해 선형적인 예측
  - (오른쪽 그래프) PolynomialFeatures(degree=10)을 사용해 먼저 데이터를 확장하고 StandardScaler를 사용해 스케일을 조정한 후 릿지 모델을 적용

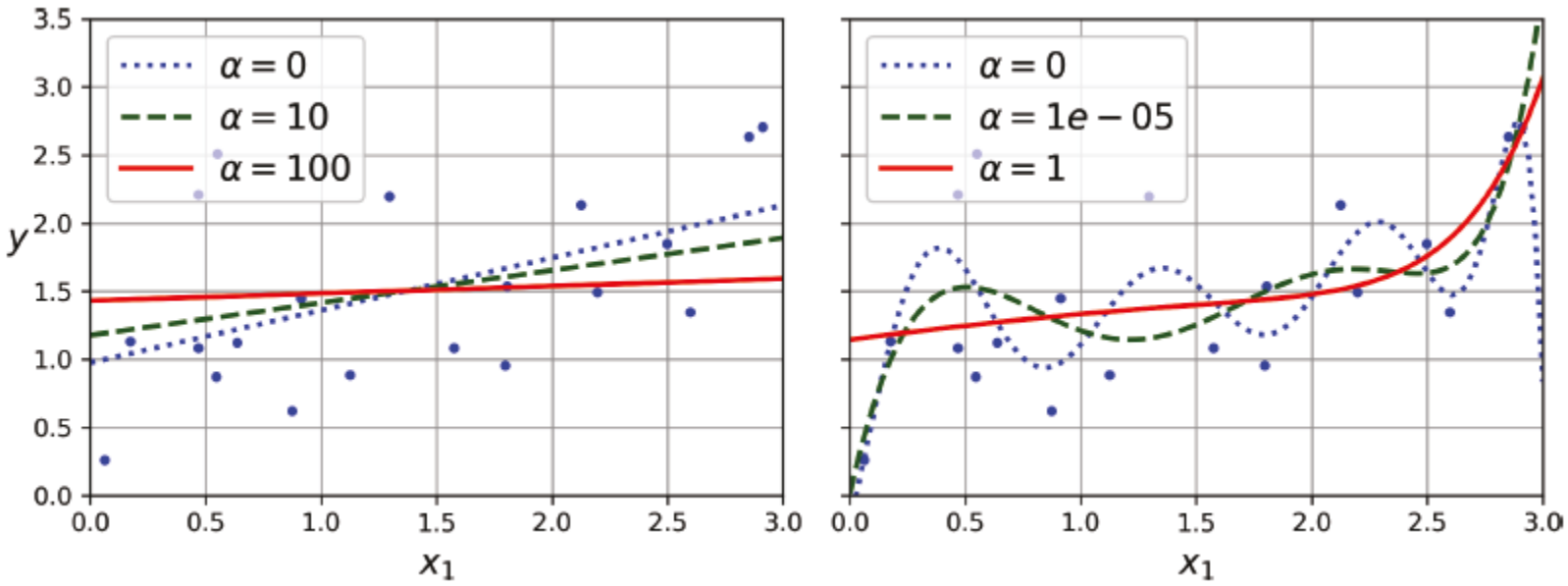

그림 4-17 다양한 수준의 릿지 규제를 사용한 선형 회귀(왼쪽)와 다항 회귀(오른쪽)

4.5 규제가 있는 선형 모델(3)

- 릿지 회귀의 정규 방정식
  - A는 편향에 해당하는 맨 왼쪽 위의 원소가 0인 (n + 1) × (n + 1)의 단위 행렬(identity matrix)

식 4-9 릿지 회귀의 정규 방정식

$$\hat{\boldsymbol{\theta}} = \left(\mathbf{X}^T\mathbf{X} + \alpha\mathbf{A}\right)^{-1}\mathbf{X}^T\mathbf{y}$$

4.5 규제가 있는 선형 모델(4)

- 사이킷런에서 정규 방정식을 사용한 릿지 회귀를 적용

```
>>> from sklearn.linear_model import Ridge
>>> ridge_reg = Ridge(alpha=0.1, solver="cholesky")
>>> ridge_reg.fit(X, y)
>>> ridge_reg.predict([[1.5]])
array([[1.55325833]])
```

- 확률적 경사 하강법을 사용

```
>>> sgd_reg = SGDRegressor(penalty="l2", alpha=0.1 / m, tol=None,
...                        max_iter=1000, eta0=0.01, random_state=42)
...
>>> sgd_reg.fit(X, y.ravel()) # fit()은 1D 타깃을 기대하므로 y.ravel()로 씁니다.
>>> sgd_reg.predict([[1.5]])
array([1.55302613])
```

4.5 규제가 있는 선형 모델(5)

4.5.2 라쏘 회귀

- 라쏘(least absolute shrinkage and selection operator, lasso) 회귀는 선형 회귀의 또 다른 규제된 버전
  - 릿지 회귀처럼 비용 함수에 규제항을 더하지만 ℓ_2 노름 대신 가중치 벡터의 ℓ_1 노름을 사용

식 4-10 라쏘 회귀의 비용 함수

$$J(\boldsymbol{\theta}) = \mathrm{MSE}(\boldsymbol{\theta}) + 2\alpha \sum_{i=1}^{n} |\theta_i|$$

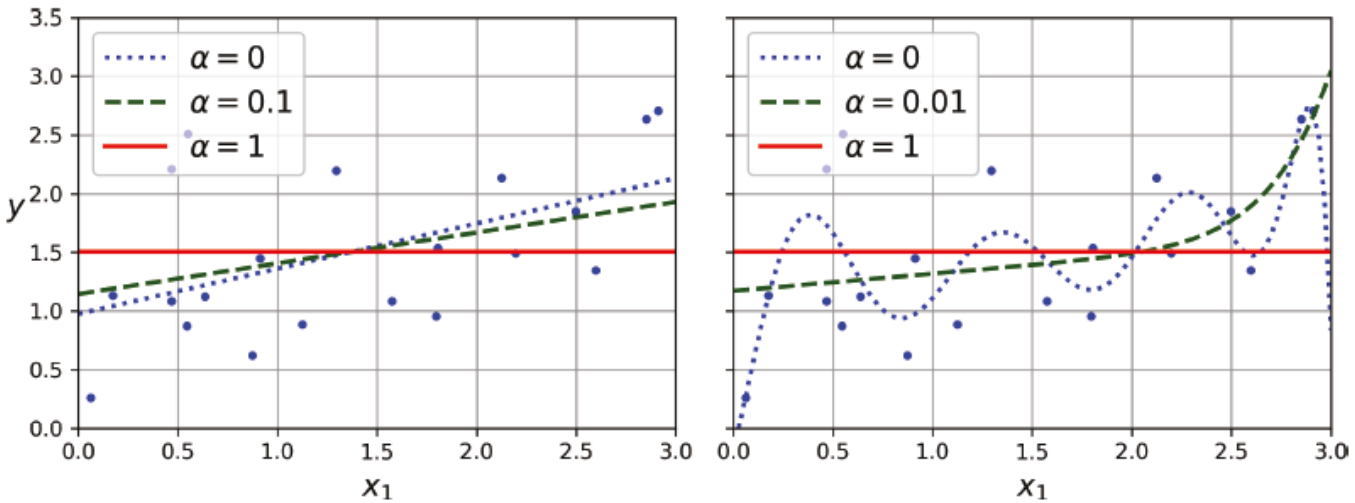

그림 4-18 다양한 수준의 라쏘 규제를 사용한 선형 회귀(왼쪽)와 다항 회귀(오른쪽)

4.5 규제가 있는 선형 모델(6)

- 라쏘 회귀는 자동으로 특성 선택을 수행하고 희소 모델(sparse model)을 생성

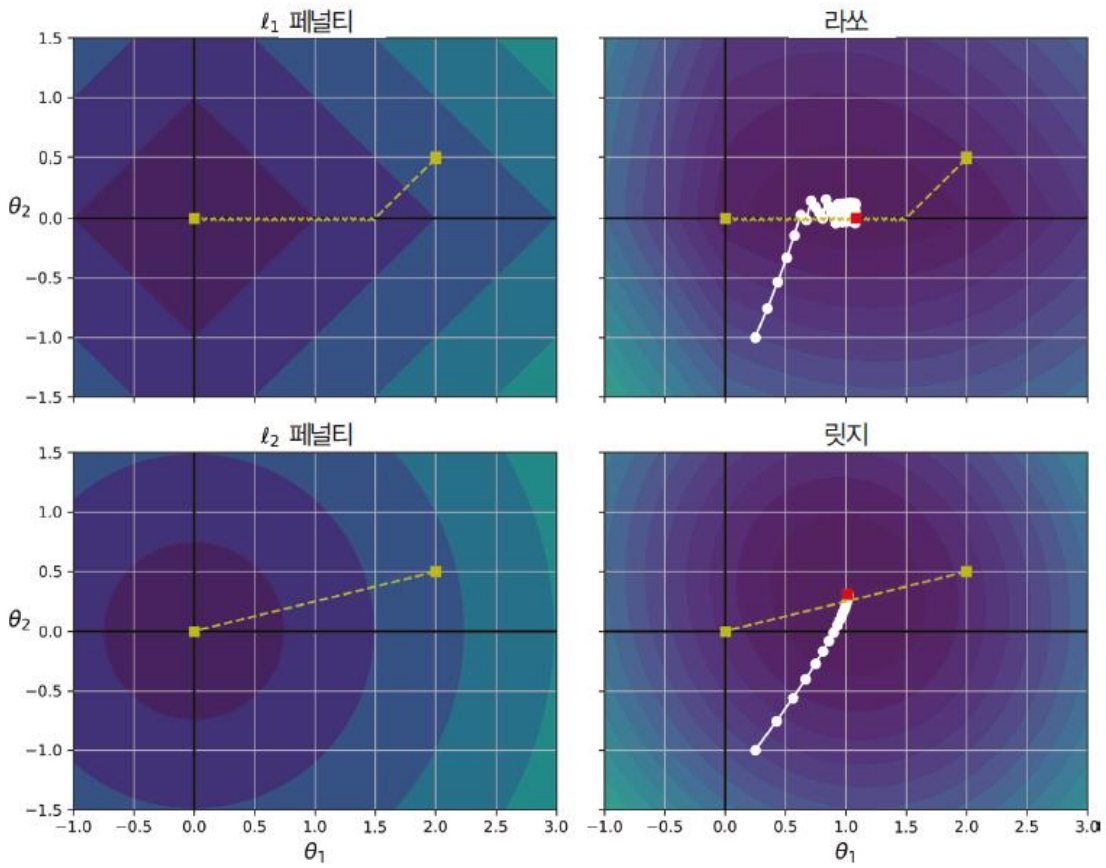

그림 4-19 라쏘 vs. 릿지 규제

4.5 규제가 있는 선형 모델(7)

- 라쏘의 비용 함수는 θ_i = 0(i= 1, 2, ..., n일 때)에서 미분 가능하지 않음
  - θ_i = 0일 때 서브그레이디언트 벡터(subgradient vector) **g**를 사용하면 경사 하강법을 적용하는 데 문제가 없음

식 4-11 라쏘 회귀의 서브그레이디언트 벡터

$$g(\boldsymbol{\theta}, J) = \nabla_{\boldsymbol{\theta}} \mathrm{MSE}(\boldsymbol{\theta}) + \alpha \begin{pmatrix} \mathrm{sign}(\theta_1) \\ \mathrm{sign}(\theta_2) \\ \vdots \\ \mathrm{sign}(\theta_n) \end{pmatrix} \quad \text{여기서} \quad \mathrm{sign}(\theta_i) = \begin{cases} -1 & \theta_i < 0\text{일 때} \\ 0 & \theta_i = 0\text{일 때} \\ +1 & \theta_i > 0\text{일 때} \end{cases}$$

- Lasso 클래스를 사용한 간단한 사이킷런 예제

```
>>> from sklearn.linear_model import Lasso
>>> lasso_reg = Lasso(alpha=0.1)
>>> lasso_reg.fit(X, y)
>>> lasso_reg.predict([[1.5]])
array([1.53788174])
```

- Lasso 대신 SGDRegressor(penalty="l1", alpha=0.1)을 사용할 수도 있음

4.5 규제가 있는 선형 모델(8)

4.5.3 엘라스틱넷

- 엘라스틱넷 회귀(elastic net regression)
  - 릿지 회귀와 라쏘 회귀를 절충한 모델
  - 규제항은 릿지와 회귀의 규제항을 단순히 더한 것이며, 혼합 정도는 혼합 비율 r을 사용해 조절
  - r = 0이면 엘라스틱넷은 릿지 회귀와 같고, r = 1이면 라쏘 회귀와 같음

식 4-12 엘라스틱넷 비용 함수

$$J(\boldsymbol{\theta}) = \mathrm{MSE}(\boldsymbol{\theta}) + r\left(2\alpha\sum_{i=1}^{n}|\theta_i|\right) + (1-r)\left(\frac{\alpha}{m}\sum_{i=1}^{n}\theta_i^2\right)$$

- 사이킷런의 ElasticNet을 사용한 간단한 예제

```
>>> from sklearn.linear_model import ElasticNet
>>> elastic_net = ElasticNet(alpha=0.1, l1_ratio=0.5)
>>> elastic_net.fit(X, y)
>>> elastic_net.predict([[1.5]])
array([1.54333232])
```

4.5 규제가 있는 선형 모델(9)

4.5.4 조기 종료

- 조기 종료(early stopping)
  - 검증 오차가 최솟값에 도달하면 바로 훈련을 중지

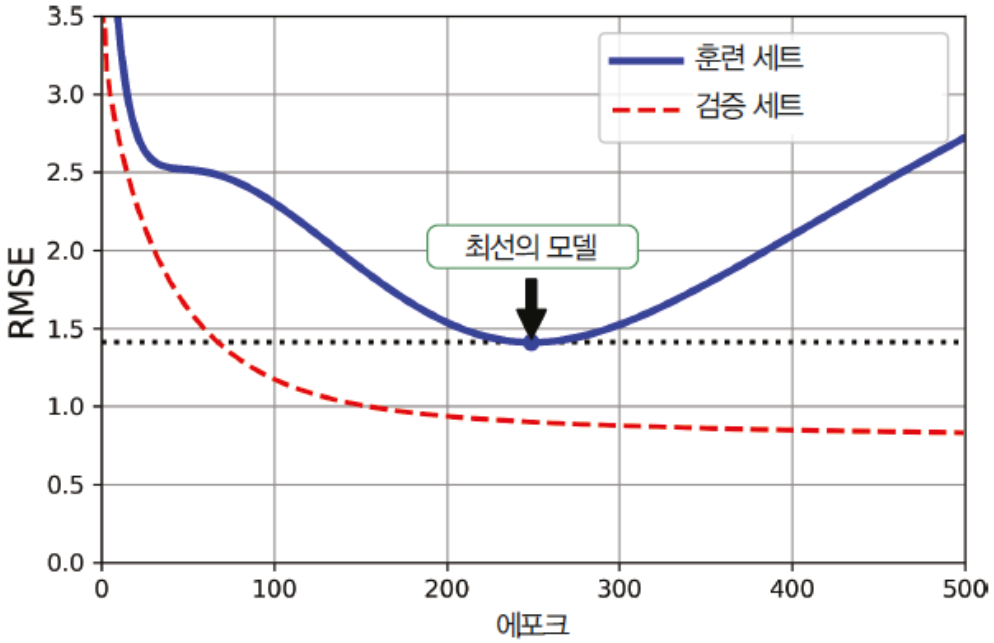

그림 4-20 조기 종료 규제

4.5 규제가 있는 선형 모델(10)

- 조기 종료 코드 예제

```
from copy import deepcopy
from sklearn.metrics import mean_squared_error
from sklearn.preprocessing import StandardScaler

X_train, y_train, X_valid, y_valid = [...] # 2차 방정식 데이터셋을 나눕니다.

preprocessing = make_pipeline(PolynomialFeatures(degree=90, include_bias=False),
                              StandardScaler())
X_train_prep = preprocessing.fit_transform(X_train)
X_valid_prep = preprocessing.transform(X_valid)
sgd_reg = SGDRegressor(penalty=None, eta0=0.002, random_state=42)
n_epochs = 500
best_valid_rmse = float('inf')

for epoch in range(n_epochs):
    sgd_reg.partial_fit(X_train_prep, y_train)
    y_valid_predict = sgd_reg.predict(X_valid_prep)
    val_error = mean_squared_error(y_valid, y_valid_predict, squared=False)
    if val_error < best_valid_rmse:
        best_valid_rmse = val_error
        best_model = deepcopy(sgd_reg)
```

- 다항 특성을 추가하고 훈련 세트와 검증 세트 모두에 대해 모든 입력 특성의 스케일을 조정
  (이 코드는 원래 훈련 세트를 더 작은 훈련 세트와 검증 세트로 분할했다고 가정)
- 규제가 없고 학습률이 작은 SGDRegressor 모델을 생성
- 훈련 반복에서는 fit() 대신 partial_fit()을 호출하여 점진적인 학습을 수행
- 각 에포크에서 검증 세트의 RMSE를 측정
- 확인된 가장 낮은 RMSE보다 낮으면 best_model 변수에 모델의 복사본을 저장
- 이 구현은 실제로 학습을 중지하지는 않지만 학습 후 최상의 모델로 되돌릴 수 있음

4.6 로지스틱 회귀(1)

4.6.1 확률 추정

- 선형 회귀 모델과 같이 로지스틱 회귀 모델은 입력 특성의 가중치 합을 계산하고 편향을 더함
  - 대신 선형 회귀처럼 바로 결과를 출력하지 않고 결괏값의 로지스틱을 출력

식 4-13 로지스틱 회귀 모델의 확률 추정(벡터 표현식)

$$\hat{p} = h_{\boldsymbol{\theta}}(\mathbf{x}) = \sigma\left(\boldsymbol{\theta}^{\mathrm{T}}\mathbf{x}\right)$$

식 4-14 로지스틱 함수

$$\sigma(t) = \frac{1}{1+\exp(-t)}$$

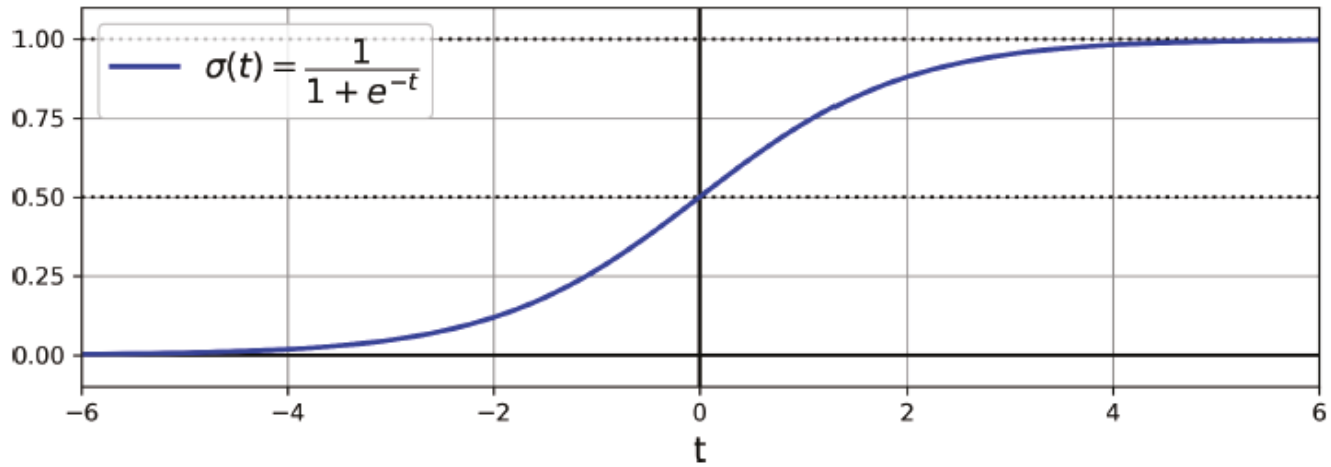

그림 4-21 로지스틱 함수

4.6 로지스틱 회귀(2)

- 로지스틱 회귀 모델이 샘플 x가 양성 클래스에 속할 확률 $\hat{p} = h_\theta(x)$ 를 추정하면 이에 대한 예측 $\hat{y}$을 쉽게 구할 수 있음

식 4-15 로지스틱 회귀 모델 예측

$$\hat{y} = \begin{cases} 0 & \hat{p} < 0.5\text{일 때} \\ 1 & \hat{p} \geq 0.5\text{일 때} \end{cases}$$

4.6 로지스틱 회귀(3)

4.6.2 훈련과 비용 함수

- 훈련의 목적
  - 양성 샘플(y = 1)에 대해서는 높은 확률을 추정하고 음성 샘플(y= 0)에 대해서는 낮은 확률을 추정하는 모델의 파라미터 벡터 **θ**를 찾는 것

식 4-16 하나의 훈련 샘플에 대한 비용 함수

$$c(\boldsymbol{\theta}) = \begin{cases} -\log(\hat{p}) & y = 1\text{일 때} \\ -\log(1-\hat{p}) & y = 0\text{일 때} \end{cases}$$

- 비용 함수
  - 로그 손실(log loss) - 전체 훈련 세트에 대한 비용 함수는 모든 훈련 샘플의 비용을 평균

식 4-17 로지스틱 회귀의 비용 함수(로그 손실)

$$J(\boldsymbol{\theta}) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log\left(\hat{p}^{(i)}\right) + \left(1-y^{(i)}\right)\log\left(1-\hat{p}^{(i)}\right)\right]$$

- 비용 함수의 편도함수

식 4-18 로지스틱 비용 함수의 편도함수

$$\frac{\partial}{\partial\theta_j}J(\boldsymbol{\theta}) = \frac{1}{m}\sum_{i=1}^{m}\left(\sigma\left(\boldsymbol{\theta}^{\mathrm{T}}\mathbf{x}^{(i)}\right) - y^{(i)}\right)x_j^{(i)}$$

4.6 로지스틱 회귀(4)

4.6.3 결정 경계

- 붓꽃(Iris) 데이터셋
  - 세 개의 품종 Iris-Setosa, Iris-Versicolor, Iris-Virginica
  - 150개 꽃잎(petal)과 꽃받침(sepal)의 너비와 길이 데이터

그림 4-22 세 종류의 붓꽃

4.6 로지스틱 회귀(5)

- 꽃잎의 너비를 기반으로 Iris-Versicolor 종을 감지하는 분류기 만들기

```
>>> from sklearn.datasets import load_iris
>>> iris = load_iris(as_frame=True)
>>> list(iris)
['data', 'target', 'frame', 'target_names', 'DESCR', 'feature_names',
 'filename', 'data_module']
>>> iris.data.head(3)
   sepal length (cm) sepal width (cm) petal length (cm) petal width (cm)
0               5.1              3.5               1.4              0.2
1               4.9              3.0               1.4              0.2
2               4.7              3.2               1.3              0.2
>>> iris.target.head(3) # 샘플을 섞지 않은 상태입니다.
0 0
1 0
2 0
Name: target, dtype: int64
>>> iris.target_names
array(['setosa', 'versicolor', 'virginica'], dtype='<U10')
```

4.6 로지스틱 회귀(6)

- 데이터를 분할하고 훈련 세트에서 로지스틱 회귀 모델을 훈련

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

X = iris.data[["petal width (cm)"]].values
y = iris.target_names[iris.target] == 'virginica'
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

log_reg = LogisticRegression(random_state=42)
log_reg.fit(X_train, y_train)
```

4.6 로지스틱 회귀(7)

- 꽃잎의 너비가 0~3cm인 꽃에 대해 모델의 추정 확률을 계산

```
X_new = np.linspace(0, 3, 1000).reshape(-1, 1) # 열 벡터로 바꿉니다.
y_proba = log_reg.predict_proba(X_new)
decision_boundary = X_new[y_proba[:, 1] >= 0.5][0, 0]

plt.plot(X_new, y_proba[:, 0], "b--", linewidth=2,
         label="Iris-Virginica가 아닐 확률")
plt.plot(X_new, y_proba[:, 1], "g-", linewidth=2, label="Iris-Virginica일 확률")
plt.plot([decision_boundary, decision_boundary], [0, 1], "k:", linewidth=2,
         label="결정 경계")
[...] # 그리드, 레이블, 축, 범례, 화살표, 샘플을 추가합니다.
plt.show()
```

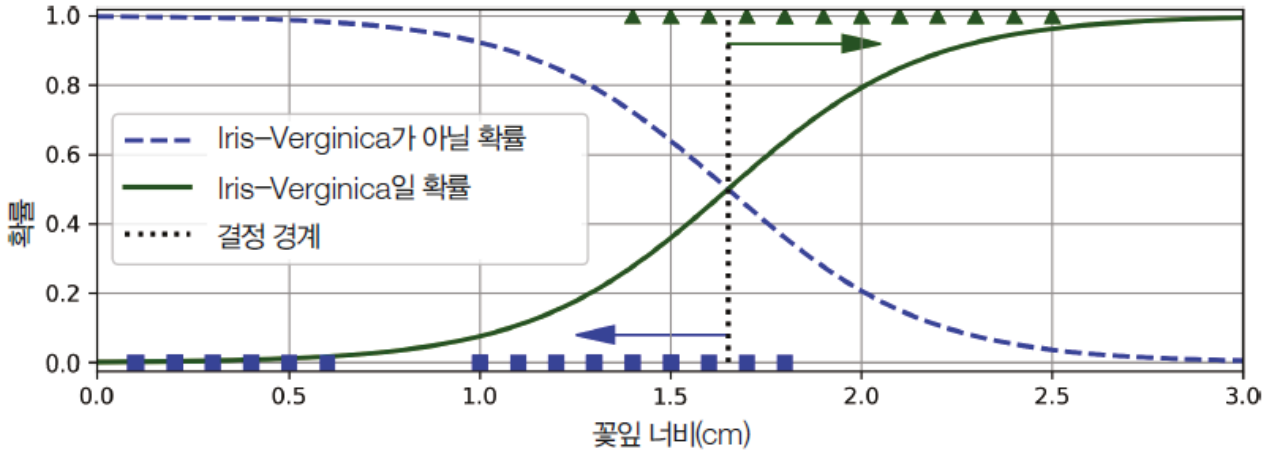

그림 4-23 추정 확률과 결정 경계

4.6 로지스틱 회귀(8)

- predict_proba() 메서드 대신 predict() 메서드를 사용하여 클래스를 예측

```
>>> decision_boundary
1.6516516516516517
>>> log_reg.predict([[1.7], [1.5]])
array([True, False])
```

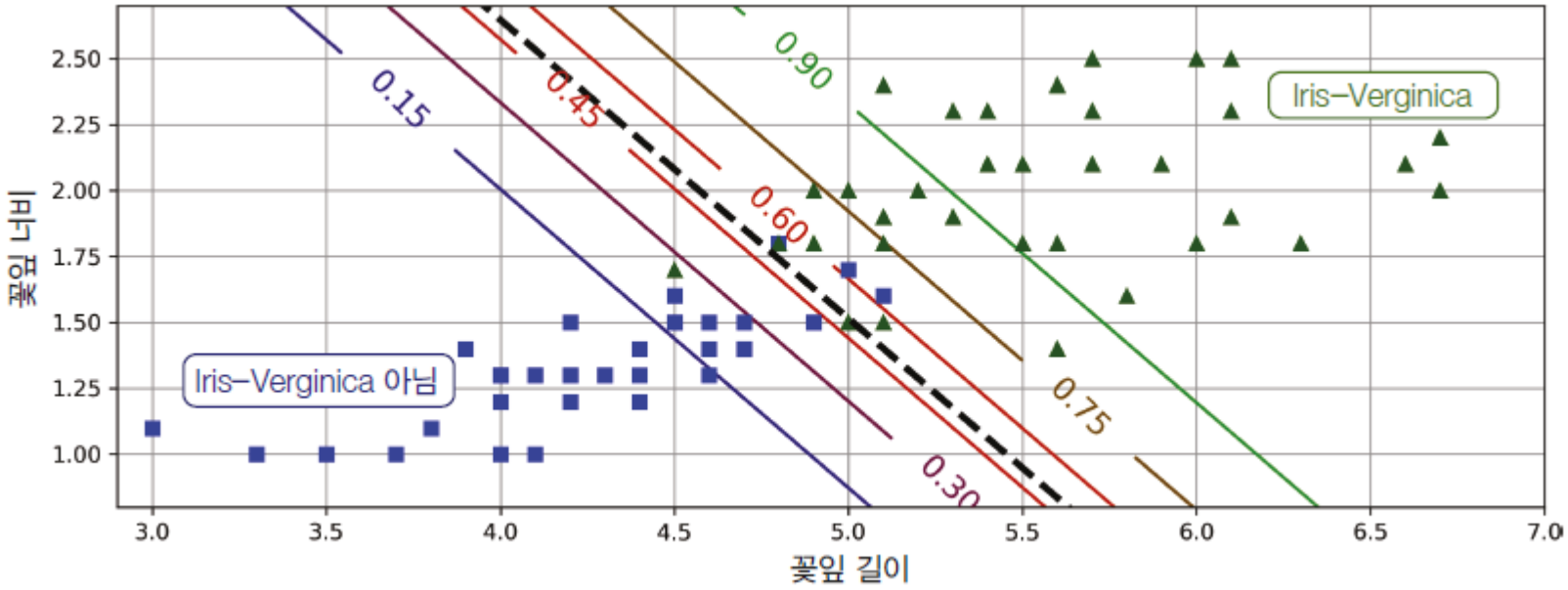

그림 4-24 선형 결정 경계

4.6 로지스틱 회귀(9)

4.6.4 소프트맥스 회귀

- 소프트맥스 회귀(softmax regression) 또는 다항 로지스틱 회귀(multinomial logistic regression)
  - 여러 개의 이진 분류기를 훈련시켜 연결하지 않고 직접 다중 클래스를 지원하도록 일반화
  - 샘플 **x**가 주어지면 먼저 소프트맥스 회귀 모델이 각 클래스 k에 대한 점수 $s_k(\mathbf{x})$를 계산하고, 그 점수에 소프트맥스 함수(또는 정규화된 지수 함수)를 적용하여 각 클래스의 확률을 추정

식 4-19 클래스 k에 대한 소프트맥스 점수

$$s_k(\mathbf{x}) = \left(\boldsymbol{\theta}^{(k)}\right)^{\mathrm{T}}\mathbf{x}$$

식 4-20 소프트맥스 함수

$$\hat{p}_k = \sigma\left(\mathbf{s}(\mathbf{x})\right)_k = \frac{\exp\left(s_k(\mathbf{x})\right)}{\sum_{j=1}^{K}\exp\left(s_j(\mathbf{x})\right)}$$

식 4-21 소프트맥스 회귀 분류기의 예측

$$\hat{y} = \underset{k}{\operatorname{argmax}}\,\sigma\left(\mathbf{s}(\mathbf{x})\right)_k = \underset{k}{\operatorname{argmax}}\,s_k(\mathbf{x}) = \underset{k}{\operatorname{argmax}}\left(\left(\boldsymbol{\theta}^{(k)}\right)^{\mathrm{T}}\mathbf{x}\right)$$

식 4-22 크로스 엔트로피 비용 함수

$$J(\boldsymbol{\Theta}) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K} y_k^{(i)}\log\left(\hat{p}_k^{(i)}\right)$$

식 4-23 클래스 k에 대한 크로스 엔트로피의 그레이디언트 벡터

$$\nabla_{\boldsymbol{\theta}^{(k)}} J(\boldsymbol{\Theta}) = \frac{1}{m}\sum_{i=1}^{m}\left(\hat{p}_k^{(i)} - y_k^{(i)}\right)\mathbf{x}^{(i)}$$

4.6 로지스틱 회귀(10)

- 소프트맥스 회귀를 사용해 붓꽃을 세 개의 클래스로 분류
  - 사이킷런의 LogisticRegression은 클래스가 둘 이상일 때 자동으로 소프트맥스 회귀를 사용

```
X = iris.data[["petal length (cm)", "petal width (cm)"]].values
y = iris["target"]
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

softmax_reg = LogisticRegression(C=30, random_state=42)
softmax_reg.fit(X_train, y_train)
```

  - 꽃잎의 길이가 5cm, 너비가 2cm인 붓꽃을 발견했다고 가정하고 이 붓꽃의 품종이 무엇인지 모델에 질의

```
>>> softmax_reg.predict([[5, 2]])
array([2])
>>> softmax_reg.predict_proba([[5, 2]]).round(2)
array([[0.  , 0.04, 0.96]])
```

4.6 로지스틱 회귀(11)

- [그림 4-25] 소프트맥스 회귀 결정 경계
  - 결정 경계를 배경색으로 구분
  - 클래스 사이의 결정 경계가 모두 선형
  - Iris-Versicolor 클래스에 대한 확률은 곡선

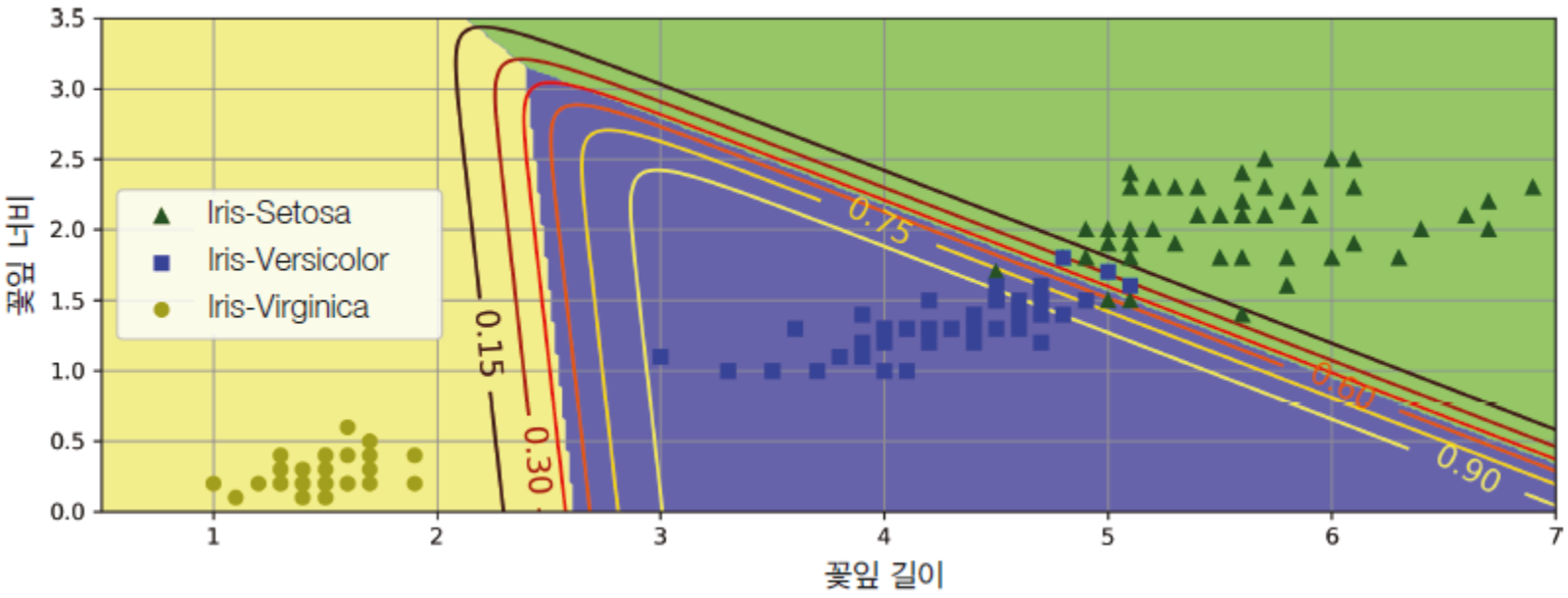

그림 4-25 소프트맥스 회귀 결정 경계

연습문제(1)

1. 수백만 개의 특성을 가진 훈련 세트에서는 어떤 선형 회귀 알고리즘을 사용할 수 있을까?
2. 훈련 세트에 있는 특성들이 각기 아주 다른 스케일을 가지고 있다. 이런 데이터에 잘 작동하지 않는 알고리즘은 무엇일까? 그 이유는 무엇일까? 이 문제를 어떻게 해결할 수 있을까?
3. 경사 하강법으로 로지스틱 회귀 모델을 훈련시킬 때 지역 최솟값에 갇힐 가능성이 있을까?
4. 충분히 오랫동안 실행하면 모든 경사 하강법 알고리즘이 같은 모델을 만들어낼까?
5. 배치 경사 하강법을 사용하고 에포크마다 검증 오차를 그래프로 나타내봤다. 검증 오차가 일정하게 상승되고 있다면 어떤 일이 일어나고 있는 걸까? 이 문제를 어떻게 해결할 수 있나?
6. 검증 오차가 상승하면 미니배치 경사 하강법을 즉시 중단하는 것이 좋은 방법인가?
7. (우리가 언급한 것 중에서) 어떤 경사 하강법 알고리즘이 가장 빠르게 최적 솔루션의 주변에 도달할까? 실제로 수렴하는 것은 어떤 것인가요? 다른 방법들도 수렴하게 만들 수 있나?
8. 다항 회귀를 사용했을 때 학습 곡선을 보니 훈련 오차와 검증 오차 사이에 간격이 크다. 무슨 일이 생긴 걸까? 이 문제를 해결하는 세 가지 방법은 무엇인가?

연습문제(2)

9. 릿지 회귀를 사용했을 때 훈련 오차와 검증 오차가 거의 비슷하고 둘 다 높았다. 이 모델에는 높은 편향이 문제인가, 아니면 높은 분산이 문제인가? 규제 하이퍼파라미터 α를 증가시켜야 할까, 아니면 줄여야 할까?

10. 다음과 같이 사용해야 하는 이유는?
    - 평범한 선형 회귀(즉, 아무런 규제가 없는 모델) 대신 릿지 회귀
    - 릿지 회귀 대신 라쏘 회귀
    - 라쏘 회귀 대신 엘라스틱넷 회귀

11. 사진을 낮과 밤, 실내와 실외로 분류하려 한다. 두 개의 로지스틱 회귀 분류기를 만들어야 할까, 아니면 하나의 소프트맥스 회귀 분류기를 만들어야 할까?

12. 사이킷런을 사용하지 않고 넘파이만 사용하여 조기 종료를 사용한 배치 경사 하강법으로 소프트맥스 회귀를 구현